Análise de Eficácia de Testes com Teste de Mutação

Disciplina: Teste de Software
Estudante: Gabriel Lucas Tinoco de Aguiar
Matrícula: 837607

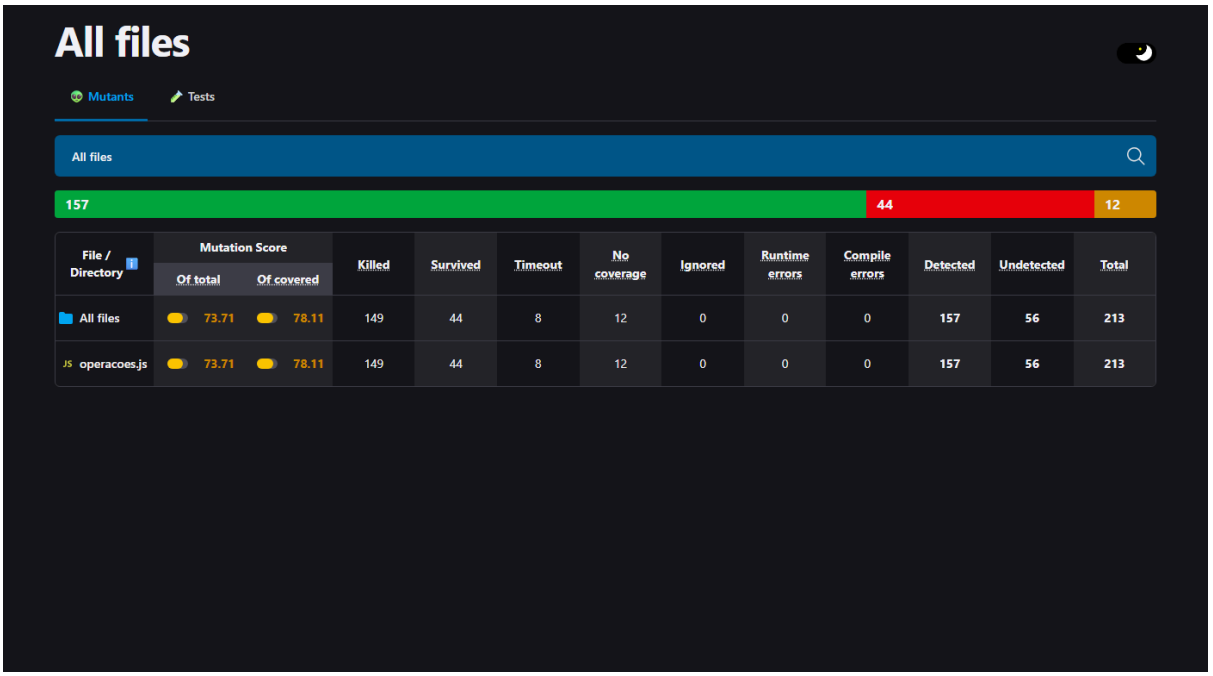
Objetivo

Este trabalho avalia a eficácia da suíte Jest da biblioteca de operações matemáticas usando o StrykerJS. A cobertura mede se o código foi executado e o teste de mutação avalia se as asserções detectam alterações que simulam defeitos.

Análise inicial

Todos os 50 testes iniciais passaram com sucesso, alcançando uma cobertura de 85,41% para statements, 58,82% para branches, 100% para funções e 98,64% para linhas. No entanto, a primeira análise do Stryker registrou um mutation score de 73,71%, com 44 mutantes sobreviventes, 8 timeouts e 12 sem cobertura. Como a execução utilizou a suíte original de testes, é normal que ocorram pequenas variações nos timeouts dependendo do limite de tempo de cada processo.

Esses resultados deixam claro por que uma cobertura alta, especialmente a de linhas, não garante a eficácia das validações. O teste inicial de fatorial, por exemplo, avaliava apenas o número positivo 4, ignorando o comportamento para entradas negativas e os casos-base 0 e 1. Da mesma forma, o teste de primalidade confirmava apenas que o 7 é primo sem testar entradas inválidas. Já o cálculo da mediana utilizava um array previamente ordenado, o que seria incapaz de revelar eventuais falhas na ordenação.



Análise de mutantes e solução

Entre os mutantes sobreviventes da execução inicial, destacaram-se:

1- Fatorial: mutações na condição de número negativo ou nos casos-base sobreviveram porque a suíte original verificava somente fatorial(4). Foram adicionadas as verificações para fatorial(0), fatorial(1) e para a mensagem de erro de fatorial(-1).

2- Primalidade: alterações no tratamento de entradas pequenas e no laço sobreviveram porque só havia um exemplo positivo. Foram incluídos isPrimo(1) === false, isPrimo(2) === true e isPrimo(4) === false.

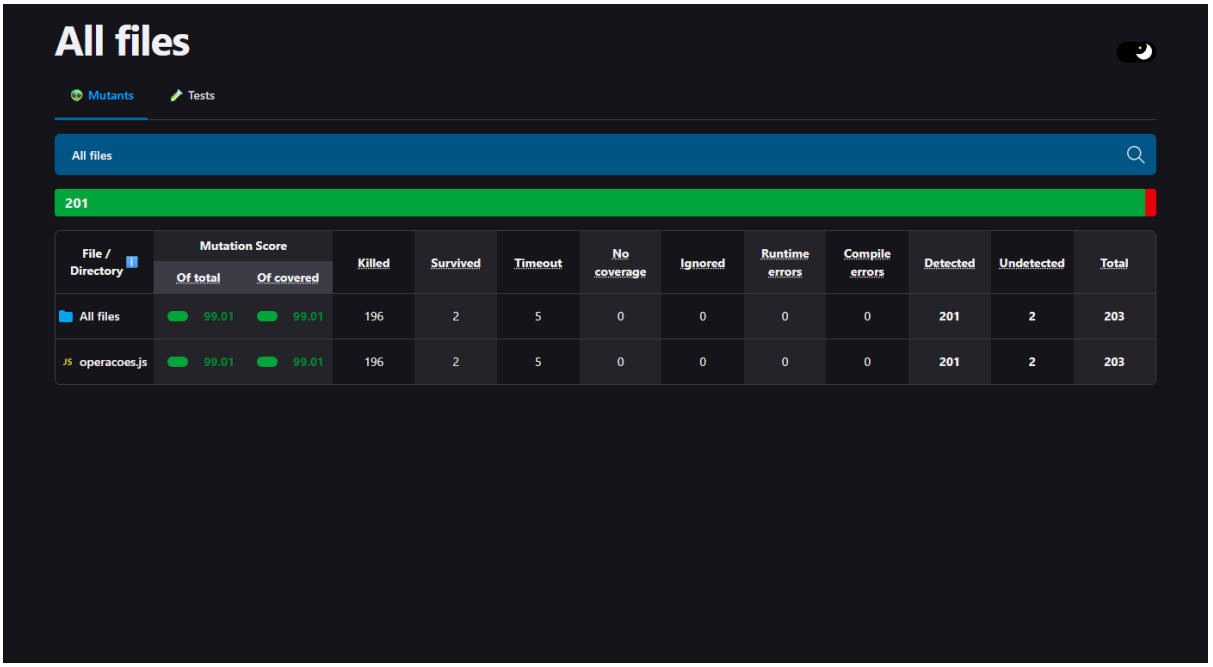
3- Mediana: mutações que removiam ou alteravam a ordenação sobreviveram porque o array original já estava ordenado. Foram adicionados arrays fora de ordem, incluindo um caso par, para verificar a ordenação e o cálculo da mediana.

A suíte de testes foi expandida para englobar erros em operações de raiz, divisão e inverso. Além disso, adicionamos validações para arrays vazios, paridade, limites do clamp, conversões com valores não nulos, divisibilidade falsa e retornos falsos em comparações.

Aproveitamos também para refatorar o código e remover condições redundantes no cálculo do fatorial e no produto de arrays vazios. Essa mudança foi feita porque os próprios acumuladores já produzem o valor 1 para esses cenários naturalmente, o que mantém o resultado correto sem a necessidade de verificações extras.

Resultados finais

Após as alterações, 59 de 59 testes passaram e a cobertura atingiu 100% em statements, branches, funções e linhas. O mutation score final foi 99,01%: 196 mutantes foram mortos, 2 sobreviveram e 5 tiveram timeout. Os dois sobreviventes são equivalentes para os resultados observáveis do clamp: substituir < por <= ou > por >= nos limites não muda o valor retornado quando o valor já é igual ao limite.



Conclusão

Embora a métrica de cobertura tenha confirmado a execução de todos os caminhos instrumentados, ela não garantia, isoladamente, que os testes saberiam diferenciar o comportamento correto de um defeito. Foi a análise de mutação que deixou essas lacunas evidentes, direcionando a criação de casos de teste focados em entradas de fronteira e resultados esperados específicos.

O salto de 73,71% para 99,01% comprova o aumento real na capacidade de detecção da suíte. Por fim, vale ressaltar que os mutantes equivalentes e os casos de timeout precisam ser interpretados de forma separada daqueles que sobreviveram apenas por ausência de asserções.